

ОС и БЛР2

Подсистема ввода-вывода

Подсистема ввода-вывода в ОС — это всё, что отвечает за работу с устройствами (диск, мышь и т.д.) **через единый интерфейс**.

Пример цепочки для чтения файла:

программа → `read()` → **VFS/ФС** → **кеш ОС (page cache)** → **драйвер** → **устройство (диск)**

Главные задачи подсистемы I/O:

- дать приложениям единый API (`open/read/write`)
- буферизация и кэширование
- планирование запросов к устройству
- обработка прерываний
- ускорение через DMA

Кеши: назначение

Зачем вообще кеш в I/O?

Потому что **RAM в разы быстрее диска**, и ОС старается:

- **не читать одно и то же с диска повторно**
- **записывать “пачками”**, а не по 1 мелкой операции

Главный кеш ОС — Page Cache

Page cache — это когда данные файлов после чтения попадают в память страницами (обычно 4KB).

Если второй раз читаешь тот же участок файла:

- **HIT** → берётся из RAM (быстро)
- **MISS** → идём на диск (медленно)

Кеш записи (write-back)

ОС может сделать так:

- `write()` вернулся “успешно”

- но физически данные на диск ещё не ушли (они в RAM как **dirty pages**)
- Сброс принудительно:
- `fsync()` / `fdatasync()`

Алгоритмы вытеснения страниц (page replacement)

Когда памяти под кеш уже мало, ОС должна решить:
какие страницы выкинуть, чтобы освободить место.

1) LRU

Выкидываем то, что **давно не использовали**.

- плюс: обычно хорошо попадает в реальность
- минус: “идеальный” LRU дорого поддерживать

2) Clock / Second chance

Практичная версия LRU.

Идея:

- у страниц есть бит “использовалась недавно”
- если да → даём второй шанс
- если нет → выкидываем

3) FIFO

Выкидываем то, что **самое старое**.

Простой, но может выкинуть полезное.

4) Optimal (MIN, Belady)

Идеальный алгоритм: выкидывает страницу, которая понадобится **позже всех**.

- даёт минимум промахов
 - в реальности невозможен (нужно знать будущее)
- Используется как **эталон для сравнения**.

5) LFU (Least Frequently Used)

Выкидывает страницу, которую использовали **реже всего** (по счётчику обращений).

- хорошо, если важны “часто используемые” данные
- “старые популярные” страницы могут зависнуть навсегда → нужен decay (старение)

Виды доступа к памяти / I/O: **O_DIRECT** и **DMA**

O_DIRECT

Это режим, когда чтение/запись идут **МИМО** page cache ОС.

То есть:

- ОС **не кеширует** файл в RAM
- запрос идёт ближе к “железу”

Зачем?

- базам данных и приложениям со своим кешем
- чтобы не было “двойного кеширования” (и ОС кеширует, и приложение)

Минусы:

- часто нужны выравнивания (alignment)
- маленькие операции могут быть медленнее (нет page cache)

DMA (Direct Memory Access)

DMA — это когда устройство может **само копировать данные** между RAM и устройством **без постоянной работы CPU**.

Как это выглядит:

1. CPU настраивает передачу (адрес, размер)
2. DMA-контроллер делает копирование сам
3. по завершению приходит **прерывание**

Плюсы DMA:

- CPU меньше занят
- выше скорость I/O
- можно параллелить вычисления и копирование

HDD vs SSD: устройство

HDD (жёсткий диск)

Внутри:

- вращающиеся магнитные пластины
- головка чтения/записи

Из-за механики есть задержки:

- **seek time** (двигать головку)

- **rotational latency** (ждать пока сектор повернётся)

👉 Поэтому HDD:

- **очень медленный на random**
- норм на sequential
- дешёвый на большие объёмы

SSD

Внутри

- NAND флэш-память
- контроллер
- 👉 SSD:
- нет механики → **random быстрый**
- но запись сложнее: стирание блоков + wear leveling + garbage collection
- есть ограничение по ресурсу записи (TBW), но обычно хватает надолго

Виды устройств ввода-вывода: мышь/диск

В Linux устройства лежат в `/dev` как спецфайлы.

Есть 2 главных типа:

Character device (символьные) — пример: мышь

Поток байт, без блоков.

Примеры:

- `/dev/input/mice`
- `/dev/input/event*`

Мышь = character device (обычно)

Block device (блочные) — пример: диск

Работают блоками (секторами), можно обращаться по адресу.

Примеры:

- `/dev/sda` , `/dev/sda1`
- `/dev/nvme0n1` , `/dev/nvme0n1p1`

Диск = block device

Что показывает команда: `ls -l /dev | grep '^[^-\ld]'`

Что делает `ls -l /dev`

`ls -l` показывает **подробный список** объектов в `/dev`.

Первый символ — тип объекта

Это самое важное для твоего вопроса:

- `-` — обычный файл
 - `d` директория (папка)
 - `l` символическая ссылка (symlink)
 - `b` **block device** (диск/раздел/блочное устройство)
 - `c` **character device** (мышь, клавиатура, терминал, устройства ввода)
- То есть
- **диск** → `b`
 - **мышь** → `c`

Зачем тут `grep '^[-ld]'`

Регулярка `^[-ld]` расшифровывается так:

- `^` = начало строки
- `[...]` = “один символ из набора”
- `^[...]` = “один символ НЕ из набора” (это *отрицание*)
- `[-ld]` = символы `-`, `l`, `d`

✅ В итоге:

👉 `^[-ld]` значит:

вывести строки, где ПЕРВЫЙ символ НЕ равен `-`, `l`, `d`

То есть мы **отсекаем**:

- `-` — обычные файлы
- `l` ссылки
- `d` папки

И **оставляем** в основном устройства

2) Файловая система

Файловая система (ФС) — это способ, как ОС **хранит файлы и папки на диске**: где лежат данные, как они называются, какие у них права, как находить нужные блоки.

Обычно ФС хранит:

- **данные файла** (content)

- **метаданные** (размер, владелец, права, время, ссылки)
- **структуру каталогов**
- **служебные структуры** (таблицы/иноды/журнал)

2.1 Виды файловых систем

По назначению

1) Дисковые (локальные)

- ext4, XFS, NTFS, FAT32, exFAT
Используются на HDD/SSD/флешках.

2) Сетевые

- NFS, SMB/CIFS
Файлы лежат на другом компьютере, доступ по сети.

3) Виртуальные (псевдо-ФС)

- /proc , /sys , /dev
Файлы “не на диске”, а генерируются ОС “на лету”.

4) Специальные/временные

- tmpfs (в RAM), swap (подкачка)

По принципу организации хранения

Инодные ФС (Unix-like) — ext4, XFS

Каждый файл описывается **inode** (метаданные + указатели на блоки данных).

FAT-подобные — FAT32/exFAT

Есть таблица размещения файлов (FAT), проще, но хуже по надёжности/возможностям.

2.2 Журналирование (journaling)

Что такое журналирование

Журналирование нужно для защиты от ситуаций типа:

вырубили свет / зависла ОС во время записи.

ФС ведёт **журнал операций**, чтобы после сбоя:

- либо **докатить изменения**
- либо **откатить** и вернуть ФС в согласованное состояние.

Главная идея:

👉 **сначала пишем в журнал, потом в основную ФС**

Это уменьшает риск “битой структуры” и долгих проверок (`fsck`).

Какие бывают режимы журналирования

1) Journal (data journaling)

- в журнал пишутся **и данные, и метаданные**
- самый надёжный
- но медленнее

2) Ordered (метаданные + порядок)

- в журнал пишутся **метаданные**
- данные пишутся “обычно”, но ОС следит, чтобы **данные успели записаться до метаданных**
- баланс скорость/надёжность (часто дефолт)

3) Writeback

- журналируются только метаданные
- порядок записи данных не гарантируется
- самый быстрый, но при сбое возможны “мусорные данные в файле”

2.3 Возможности ОС по работе с ФС: системные вызовы

Файловая система в Linux/Unix управляется через системные вызовы.

Основные syscall'ы (что надо знать)

Открытие/закрытие

- `open()` — открыть файл и получить fd (file descriptor)
- `close()` — закрыть fd

Чтение/запись

- `read()` — чтение байтов
- `write()` — запись байтов

Перемещение позиции

- `lseek()` — сдвиг указателя чтения/записи (random access)

Метаданные

- `stat()` / `fstat()` — информация о файле (размер, права, даты)
- `chmod()` — права
- `chown()` — владелец
- `umask()` — маска прав по умолчанию

Каталоги

- `mkdir()` — создать папку
- `rmdir()` — удалить пустую папку
- `opendir()/readdir()` (внутри через `syscall`'ы) — обход файлов

Работа с файлами как объектами

- `link()` — создать жёсткую ссылку (hard link)
- `unlink()` — удалить ссылку на файл
- `rename()` — переименование/перемещение
- `symlink()` — символическая ссылка

Надёжная запись на диск

- `fsync()` / `fdatasync()` — заставить реально сбросить данные на диск

Важный момент про дескриптор

После `open()` ОС возвращает **fd** — число типа `3, 4, 5...`

`fd` — это “ручка” на открытый файл, дальше все операции идут через него:

- `read(fd, ...)`
- `write(fd, ...)`
- `fsync(fd)`

2.4 Отображение файла в память (memory-mapped file)

Это про `mmap()`.

Что такое `mmap()`

`mmap()` позволяет **сделать файл частью виртуальной памяти процесса**.

То есть ты работаешь с файлом как с массивом в памяти:

- читаешь/пишешь просто обращением к адресу
- ОС сама подгружает нужные страницы файла в RAM (page fault)

Как оно работает (логика)

1. процесс вызывает `mmap()`
2. ОС “привязывает” участок файла к диапазону виртуальных адресов
3. при обращении к странице, которой нет → **page fault**
4. ОС читает нужный блок файла в RAM (page cache) и продолжает выполнение

Плюсы mmap

- удобно и быстро для больших файлов
- ОС сама делает кэширование страниц
- меньше `syscall`’ов (не нужно постоянно `read()`)

Минусы mmap

- ошибки могут быть как `segfault` при доступе
- нужно аккуратно синхронизировать запись (`msync()`)
- для маленьких файлов часто проще `read/write`

Разница `read()` vs `mmap()`

`read()`:

- ты явно копируешь данные “в буфер”
- каждый вызов — `syscall`

`mmap()`:

- файл “как память”
- ОС сама подкачивает страницы по мере обращения

3) Отказоустойчивость (ошибки доступа, RAID, ...)

3.1 Что такое отказоустойчивость

Отказоустойчивость — способность системы **продолжать работать при сбоях** (например, при выходе диска, ошибках чтения, сбоях питания).

Часто рядом используют термины:

- **Надёжность (reliability)** — как редко ломается
- **Доступность (availability)** — как долго система “в строю” (время работы без простоя)

3.2 Ошибки доступа (I/O errors): какие бывают и что они значат

Типичные ошибки на уровне диска/ОС

1) Ошибка чтения (read error)

Диск не смог прочитать блок (повреждение сектора, проблемы с головкой/контроллером, деградация SSD).

2) Ошибка записи (write error)

Запись не удалась (проблемы с носителем, контроллером, питание, износ SSD).

3) Таймаут / device reset

Устройство “тормозит” или временно пропадает → драйвер делает сброс.

4) Повреждение данных (silent data corruption)

Самое опасное: операция “успешна”, но **данные битые**. Для защиты нужны контрольные суммы (checksum), ECC и т.д.

Как ОС обычно реагирует

- **повторяет запрос (retry)** (часто помогает при временных сбоях)
- может пометить устройство как проблемное и выдать **EIO**
- файловая система может перейти в **read-only** режим при критичных ошибках
- журналирование помогает восстановить целостность структуры ФС после падения

3.3 RAID: зачем нужен

RAID — технология объединения нескольких дисков в один “логический диск” для:

- **скорости**
- **надёжности (избыточность)**
- или обеих целей сразу

Важно: **RAID ≠ backup**

RAID спасает от **отказа диска**, но не спасает от:

- удаления файлов пользователем
- вирусов
- повреждения данных на уровне файлов
- пожара/кражи

3.4 Основные RAID уровни (0–6 + популярные комбинации)

RAID 0 (striping)

- данные режутся на полосы и пишутся на несколько дисков
 - ✓ очень быстро
 - ✗ отказ 1 диска = потеря всего массива (нет отказоустойчивости)

RAID 1 (mirroring)

- полная копия данных на 2+ дисках
 - ✓ переживает отказ одного диска
 - ✓ чтение может быть быстрее (читаем с любого)
 - ✗ полезный объём = ~50%

RAID 2 (редко)

- исторически на битовом уровне + ECC
Практически не используют (заменяли RAID 5/6 и современная коррекция ошибок).

RAID 3 (редко)

- данные полосами + **отдельный диск чётности**
 - ✓ хорош для больших последовательных потоков
 - ✗ диск чётности становится узким местом → редко используется

RAID 4 (почти не используют)

- как RAID 3, но полосы крупнее (block-level) и **1 диск parity**
 - ✗ parity-диск перегружается на записи → поэтому вытеснен RAID 5

RAID 5 (striping + parity распределена)

- минимум 3 диска
- чётность **размазана по всем дискам**
 - ✓ переживает отказ **1 диска**
 - ✓ хороший баланс объём/надёжность
 - ✗ при записи есть “штраф” (надо читать+пересчитать parity)

RAID 6 (двойная parity)

- минимум 4 диска
 - ✓ переживает отказ **2 дисков**
 - ✓ лучше для больших массивов и больших дисков
 - ✗ запись ещё тяжелее чем RAID 5 (двойная чётность)

RAID 10 (1+0) — “золотой стандарт”

- зеркала + striping
 - ✓ быстро
 - ✓ отказоустойчиво (можно пережить отказ нескольких дисков **если не в одной паре**)
 - ✗ полезный объём ≈ 50%

3.5 Важные понятия по RAID (часто спрашивают)

Degraded mode

Если один диск умер — RAID 5/6/10 может работать дальше в режиме **degraded**, но:

- скорость ниже
- риск выше

Rebuild (восстановление)

После замены диска RAID “пересобирает” данные:

- RAID1/10 — копирование с зеркала
- RAID5/6 — пересчёт по чётности
 - ⚠ Rebuild долгий и нагружает массив, есть риск второй ошибки.

Write penalty (штраф записи)

В RAID5/6 запись маленького блока требует доп. операций:

- прочитать старые данные
- прочитать старую чётность
- посчитать новую чётность
- записать данные + parity → поэтому мелкая запись медленнее, чем чтение.

4) Таблицы разделов (MBR, GPT)

Таблица разделов отвечает за:

- ✓ где на диске начинаются/заканчиваются разделы
- ✓ какие они (тип, флаги, загрузочный)
- ✓ как ОС видит структуру диска

4.1 MBR (Master Boot Record)

Что это

MBR — старая схема разметки диска (BIOS эпоха).

MBR хранится в **первом секторе диска** и содержит:

- код загрузчика (boot code)
- таблицу разделов

Ограничения MBR

- ✗ максимум **4 primary раздела**
(решали через extended + logical)
- ✗ ограничение по размеру диска примерно **2 ТБ** (при 512В секторах)
- ✗ слабая защита от повреждений (нет нормальных проверок целостности)

4.2 GPT (GUID Partition Table)

Что это

GPT — современная схема (обычно вместе с **UEFI**).

Фичи GPT:

- ✓ очень много разделов (обычно 128 по умолчанию)
- ✓ поддержка огромных дисков (практически без “2 ТБ лимита”)
- ✓ есть **контроль целостности (CRC)**
- ✓ есть **резервная копия таблицы** в конце диска

Protective MBR

На GPT-диске в начале всё равно есть “защитный MBR”, чтобы старые утилиты не думали, что диск пустой.

4.3 Главное сравнение MBR vs GPT (как любят на зачёте)

MBR

- старый, BIOS
- 4 primary
- ~2ТБ лимит
- нет нормальной защиты

GPT

- новый, UEFI
- много разделов
- большие диски
- CRC + резервная таблица

4.4 Важный момент про загрузку (коротко)

- **BIOS** обычно грузится с **MBR**

- **UEFI** обычно грузится с **GPT** через **EFI System Partition (ESP)**